

MUNICIPALIDAD DE FLORES

Cantón de Flores, Heredia, Costa Rica

MANUAL TÉCNICO

Sistema de Gestión de Acuerdos Municipales

Versión 1.0 • 2026

Proyecto de Graduación — Tesis 2026

Confidencial — Uso Interno

1. Introducción

El presente Manual Técnico documenta la arquitectura, configuración, instalación y mantenimiento del Sistema de Gestión de Acuerdos Municipales desarrollado para la Municipalidad de Flores, Heredia, Costa Rica. El documento está dirigido a personal técnico con conocimientos en desarrollo de software, administración de bases de datos y administración de servidores.

El sistema fue desarrollado como proyecto de tesis 2026, con el objetivo de digitalizar y centralizar el proceso de registro, seguimiento y auditoría de los acuerdos tomados en sesiones del Concejo Municipal.

1.1 Propósito del Sistema

Automatizar el registro, consulta y seguimiento de acuerdos municipales, garantizando trazabilidad completa mediante un historial de auditoría, gestión de documentos PDF asociados y un semáforo de estados que permite visualizar el cumplimiento de cada acuerdo.

1.2 Alcance

- Registro y consulta de acuerdos y actas municipales.
- Carga y visualización de documentos PDF (actas y oficios).
- Seguimiento de estados: Pendiente, Cumplido, Vencido con semáforo visual.
- Historial de auditoría completo con registro por usuario.
- Gestión de usuarios con roles: Administrador y Usuario.
- Actualización automática de acuerdos vencidos mediante tarea programada (cron).
- Eliminación lógica (soft delete) de acuerdos y actas para preservar trazabilidad.

2. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas desacopladas: frontend (cliente), backend (API REST) y base de datos. La comunicación entre capas se realiza mediante HTTP con autenticación JWT.

2.1 Diagrama General

Capa	Tecnología	Función
Frontend	React 18 + Vite + Tailwind CSS	Interfaz de usuario (SPA)
Backend	Node.js 20 + Express 4.x	API REST — lógica de negocio
Base de Datos	MySQL 8.0	Persistencia de datos y auditoría
Autenticación	JWT (jsonwebtoken)	Sesiones seguras sin estado
Almacenamiento	Sistema de archivos local	PDFs de actas y oficios

2.2 Stack Tecnológico Detallado

2.2.1 Frontend

Paquete	Versión	Uso
React	18.x	Biblioteca principal de UI
Vite	5.x	Bundler y servidor de desarrollo
Tailwind CSS	3.x	Estilos utilitarios
react-router-dom	6.x	Enrutamiento del lado del cliente
axios	1.x	Cliente HTTP con interceptores JWT
react-hot-toast	2.x	Notificaciones emergentes
react-icons (MD)	5.x	Iconografía Material Design

2.2.2 Backend

Paquete	Versión	Uso
Node.js	20 LTS	Entorno de ejecución
Express	4.x	Framework HTTP
mysql2	3.x	Driver MySQL con soporte Promise
jsonwebtoken	9.x	Generación y validación JWT
bcryptjs	2.x	Hash seguro de contraseñas
multer	1.x	Carga de archivos PDF

Paquete	Versión	Uso
helmet	7.x	Headers de seguridad HTTP
express-rate-limit	7.x	Limitación de peticiones
express-validator	7.x	Validación de entradas
node-cron	3.x	Tareas programadas
dotenv	16.x	Variables de entorno

3. Estructura del Proyecto

3.1 Backend

```
backend/
├── config/
│   ├── db.js           # Pool de conexiones MySQL
│   └── cron.js         # Tarea programada de vencimientos
├── controllers/
│   ├── authController.js # Login y autenticación
│   ├── actasController.js # CRUD de actas
│   ├── acuerdosController.js # CRUD de acuerdos y oficios
│   └── usuariosController.js # CRUD de usuarios
├── middleware/
│   ├── auth.js         # Verificación JWT y roles
│   ├── upload.js       # Configuración Multer (actas)
│   └── validaciones.js  # Reglas express-validator
├── routes/
│   ├── auth.js         # POST /api/auth/login
│   ├── actas.js        # CRUD /api/actas
│   ├── acuerdos.js     # CRUD /api/acuerdos
│   └── usuarios.js      # CRUD /api/usuarios
├── uploads/
│   ├── actas/          # PDFs de actas
│   └── oficios/        # PDFs de oficios
├── server.js           # Punto de entrada
└── .env                # Variables de entorno
```

3.2 Frontend

```
frontend/src/
├── api/
│   └── axios.js        # Cliente HTTP con interceptores
├── components/
│   ├── Navbar.jsx     # Barra de navegación
│   ├── Semaforo.jsx   # Indicador de estado visual
│   ├── ModalConfirmar.jsx # Modal reutilizable de confirmación
│   └── ProtectedRoute.jsx # Guards de rutas por rol
├── context/
│   └── AuthContext.jsx # Estado global de sesión
├── pages/
│   ├── Login.jsx      # Inicio de sesión
│   ├── Dashboard.jsx  # Panel de control
│   ├── Acuerdos.jsx   # Listado y gestión de acuerdos
│   └── FormAcuerdo.jsx # Crear/editar acuerdo y oficios
```

```
| | └─ Actas.jsx          # Listado y gestión de actas
| | └─ Historial.jsx      # Historial por acuerdo
| | └─ HistorialGlobal.jsx# Historial del sistema completo
| | └─ Usuarios.jsx       # Gestión de usuarios (Admin)
└─ App.jsx                # Rutas principales
└─ main.jsx               # Punto de entrada
```

4. Base de Datos

El motor de base de datos utilizado es MySQL 8.0 con charset utf8mb4 y collation utf8mb4_spanish_ci, lo que garantiza soporte completo para caracteres especiales del idioma español.

4.1 Tablas Principales

Tabla	Descripción
usuarios	Almacena las cuentas de acceso al sistema con roles y estado activo/inactivo.
actas	Registra las actas de sesiones del Concejo Municipal con su PDF asociado.
acuerdos	Almacena los acuerdos tomados en cada acta, con estado, plazo y vencimiento.
oficios	PDFs de oficios asociados a cada acuerdo (relación N:1 con acuerdos).
historial_cambios	Registro de auditoría automático de todas las operaciones INSERT, UPDATE y DELETE.

4.2 Campos de Soft Delete

Las tablas acuerdos y actas implementan eliminación lógica (soft delete) para preservar la trazabilidad. Los registros nunca se borran físicamente de la base de datos.

Campo	Tipo	Descripción
eliminado	BOOLEAN	FALSE = activo, TRUE = eliminado lógicamente
fecha_eliminacion	TIMESTAMP NULL	Fecha y hora de la eliminación lógica
eliminado_por	INT NULL (FK)	ID del usuario que realizó la eliminación

4.3 Triggers

Trigger	Evento	Función
acuerdos_before_insert	BEFORE INSERT	Calcula fecha_vencimiento según plazo_dias
acuerdos_before_update	BEFORE UPDATE	Recalcula fecha_vencimiento al modificar
acuerdos_after_insert	AFTER INSERT	Registra creación en historial_cambios
acuerdos_after_update	AFTER UPDATE	Registra modificación o eliminación lógica en historial
oficios_after_insert	AFTER INSERT	Registra adición de oficio en historial
oficios_after_delete	AFTER DELETE	Registra eliminación de oficio en historial

4.4 Vistas

Vista	Descripción
vista_dashboard_estados	Conteo de acuerdos por estado para el panel de control. Excluye eliminados.
vista_acuerdos_semaforo	Acuerdos activos con color de semáforo calculado (ROJO/AMARILLO/VERDE).
vista_acuerdos_completos	Acuerdos con datos del acta y usuario registrador. Excluye eliminados.

4.5 Variable de Sesión para Auditoría

Los triggers utilizan la variable de sesión MySQL `@usuario_activo` para registrar qué usuario realizó cada operación. El backend la establece antes de cada operación mediante:

```
SET @usuario_activo = <id_usuario>;
```

Esta variable se establece en cada conexión extraída del pool antes de ejecutar cualquier consulta de escritura.

5. API REST

La API REST expone los siguientes endpoints bajo la ruta base `http://localhost:3000/api`. Todos los endpoints (excepto `/api/auth/login`) requieren el encabezado de autorización con JWT válido.

5.1 Autenticación

Método	Ruta	Auth	Descripción
POST	<code>/api/auth/login</code>	No	Inicio de sesión. Retorna JWT.

5.2 Actas

Método	Ruta	Auth	Descripción
GET	<code>/api/actas</code>	JWT	Listar actas activas (filtros opcionales)
POST	<code>/api/actas</code>	JWT	Crear acta con PDF (multipart/form-data)
GET	<code>/api/actas/:id/pdf</code>	JWT	Servir PDF del acta
PUT	<code>/api/actas/:id</code>	JWT	Editar tipo_sesion y fecha_sesion
DELETE	<code>/api/actas/:id</code>	JWT	Soft delete del acta

5.3 Acuerdos

Método	Ruta	Auth	Descripción
GET	<code>/api/acuerdos</code>	JWT	Listar acuerdos (búsqueda y filtros)
GET	<code>/api/acuerdos/dashboard</code>	JWT	Datos del panel de control
POST	<code>/api/acuerdos</code>	JWT	Crear nuevo acuerdo
PUT	<code>/api/acuerdos/:id</code>	JWT	Actualizar acuerdo
DELETE	<code>/api/acuerdos/:id</code>	JWT	Soft delete de acuerdo
GET	<code>/api/acuerdos/:id/historial</code>	JWT	Historial del acuerdo
GET	<code>/api/acuerdos/:id/oficios</code>	JWT	Listar oficinas del acuerdo
POST	<code>/api/acuerdos/:id/oficios</code>	JWT	Subir oficina PDF
DELETE	<code>/api/acuerdos/:id/oficios/:oid</code>	JWT	Eliminar oficina
GET	<code>/api/acuerdos/:id/oficios/:oid/pdf</code>	JWT	Ver PDF de oficina

5.4 Usuarios

Método	Ruta	Auth	Descripción
GET	/api/usuarios	JWT + Admin	Listar todos los usuarios
POST	/api/usuarios	JWT + Admin	Crear usuario
PUT	/api/usuarios/:id	JWT + Admin	Editar usuario (datos y contraseña)
PUT	/api/usuarios/:id/estado	JWT + Admin	Activar / desactivar usuario

5.5 Historial Global y Mantenimiento

Método	Ruta	Auth	Descripción
GET	/api/historial	JWT	Últimos 500 registros del historial global
GET	/api/admin/actualizar-vencidos	JWT	Fuerza actualización de acuerdos vencidos
GET	/api/health	No	Verificar estado del servidor

6. Seguridad

6.1 Autenticación JWT

El sistema utiliza JSON Web Tokens (JWT) para la autenticación sin estado. El token se genera al iniciar sesión y expira en 2 horas (configurable en .env). Cada solicitud protegida debe incluir el token en el encabezado HTTP:

```
Authorization: Bearer <token>
```

Para la visualización de PDFs en nueva pestaña del navegador (donde no es posible enviar encabezados), el token se acepta también como parámetro de consulta (?token=...). Este mecanismo es exclusivo para las rutas de servicio de archivos PDF.

6.2 Hash de Contraseñas

Las contraseñas se almacenan utilizando bcrypt con un factor de trabajo de 12 rondas. Nunca se almacena la contraseña en texto plano. La verificación se realiza comparando el hash almacenado con el hash del intento de inicio de sesión.

6.3 Headers de Seguridad

El middleware Helmet.js configura automáticamente los siguientes encabezados HTTP de seguridad: Content-Security-Policy, X-Frame-Options, X-XSS-Protection, Strict-Transport-Security, X-Content-Type-Options, entre otros.

6.4 Rate Limiting

Scope	Ventana	Límite	Propósito
General (/api/*)	1 hora	5,000 req/IP	Uso normal de oficina
Login (/api/auth/login)	15 minutos	20 intentos/IP	Prevenir fuerza bruta

6.5 Validación de Entradas

Todas las entradas del usuario se validan en el backend utilizando express-validator antes de procesarse. Las validaciones incluyen: formato de correo electrónico, longitud mínima de contraseña, valores de ENUM permitidos, formato de fechas y tipos de datos numéricos.

6.6 Control de Acceso por Rol

El sistema implementa dos roles con diferentes niveles de acceso:

Rol	Permisos
Usuario	Acceso a Dashboard, Acuerdos, Actas e Historial. Puede crear, editar, eliminar acuerdos y actas, y gestionar oficios.
Administrador	Todos los permisos del rol Usuario más acceso a la gestión completa de usuarios del sistema.

7. Guía de Instalación

7.1 Requisitos Previos

Software	Versión mínima	Descarga
Node.js	20 LTS	https://nodejs.org
MySQL	8.0	https://dev.mysql.com/downloads
MySQL Workbench	8.0 (recomendado)	https://dev.mysql.com/downloads/workbench
Git	Cualquiera	https://git-scm.com

7.2 Configuración de la Base de Datos

Paso 1: Abrir MySQL Workbench y conectarse al servidor MySQL local.

Paso 2: Abrir el archivo `sistema_acuerdos_v3.sql` y ejecutar el script completo (Ctrl+Shift+Enter). El script realizará las siguientes acciones:

- Crear la base de datos `sistema_acuerdos` con `charset utf8mb4`.
- Crear todas las tablas con sus relaciones y restricciones.
- Configurar los triggers de auditoría y cálculo de vencimientos.
- Crear las vistas para dashboard y semáforo.

Paso 3: Crear el usuario administrador inicial ejecutando:

```
USE sistema_acuerdos;
INSERT INTO usuarios (nombre, apellidos, correo, password_hash, rol, puesto)
VALUES ('Admin', 'Sistema', 'admin@flores.go.cr',
        '$2a$12$<hash_bcrypt>', 'Administrador', 'Administrador del
Sistema');
```

El hash de contraseña debe generarse con `bcrypt` (12 rondas). Puede utilizarse la herramienta en línea <https://bcrypt-generator.com> o ejecutar el siguiente fragmento de Node.js:

```
node -e "require('bcryptjs').hash('SuContraseña', 12, (e,h) =>
console.log(h))"
```

7.3 Configuración del Backend

Paso 1: Navegar a la carpeta del backend e instalar dependencias:

```
cd backend
npm install
```

Paso 2: Crear el archivo `.env` en la raíz del backend con el siguiente contenido:

```
PORT=3000
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=<su_contraseña_mysql>
DB_NAME=sistema_acuerdos
```

```
JWT_SECRET=<clave_aleatoria_larga_minimo_64_caracteres>  
JWT_EXPIRES_IN=2h  
UPLOADS_PATH=./uploads/actas
```

Paso 3: Crear las carpetas para almacenamiento de archivos:

```
mkdir -p uploads/actas uploads/oficios
```

Paso 4: Iniciar el servidor:

```
npm run dev      # Modo desarrollo (nodemon)  
node server.js   # Modo producción
```

7.4 Configuración del Frontend

Paso 1: Navegar a la carpeta del frontend e instalar dependencias:

```
cd frontend  
npm install
```

Paso 2: Verificar que el archivo src/api/axios.js apunta a la URL correcta del backend:

```
baseUrl: 'http://localhost:3000/api'
```

Paso 3: Iniciar el servidor de desarrollo:

```
npm run dev
```

El frontend estará disponible en <http://localhost:5173>. Para producción, ejecutar `npm run build` y servir la carpeta `dist` con cualquier servidor web estático (Nginx, Apache, etc.).

8. Mantenimiento y Operación

8.1 Tarea Programada (Cron)

El sistema ejecuta automáticamente una tarea diaria a medianoche (zona horaria America/Costa_Rica) que actualiza el estado de los acuerdos Pendientes con fecha de vencimiento pasada a Vencido. La tarea también se ejecuta al iniciar el servidor para corregir cualquier vencimiento no procesado.

Para forzar la actualización manualmente durante pruebas o mantenimiento, realizar la siguiente solicitud HTTP autenticada:

```
GET http://localhost:3000/api/admin/actualizar-vencidos
Authorization: Bearer <token_jwt>
```

8.2 Gestión de Archivos PDF

Los archivos PDF se almacenan en el sistema de archivos del servidor en dos ubicaciones:

- uploads/actas/ — PDFs de actas, con nombre igual al numero_acta (ej: ACTA-2026-001.pdf).
- uploads/oficios/ — PDFs de oficios, con nombre oficio-{numero_acuerdo}-{timestamp}.pdf.

Se recomienda configurar copias de seguridad periódicas de estas carpetas junto con la base de datos para garantizar la integridad del sistema.

8.3 Respaldo de la Base de Datos

Ejecutar el siguiente comando para generar un respaldo completo de la base de datos:

```
mysqldump -u root -p sistema_acuerdos > respaldo_$(date +%Y%m%d).sql
```

Se recomienda programar este respaldo de forma automática mediante una tarea cron del sistema operativo con frecuencia diaria.

8.4 Recuperación de Registros Eliminados

Dado que el sistema utiliza soft delete, los registros eliminados permanecen en la base de datos y pueden recuperarse ejecutando:

```
-- Recuperar un acuerdo eliminado
UPDATE acuerdos SET eliminado = FALSE, fecha_eliminacion = NULL,
    eliminado_por = NULL WHERE numero_acuerdo = 'X';

-- Recuperar un acta eliminada
UPDATE actas SET eliminado = FALSE, fecha_eliminacion = NULL,
    eliminado_por = NULL WHERE numero_acta = 'X';
```

8.5 Monitoreo

El endpoint de salud del servidor permite verificar que el servicio está operativo:

```
GET http://localhost:3000/api/health
```

Respuesta esperada:

```
{ "estado": "ok", "timestamp": "2026-01-01T00:00:00.000Z" }
```

9. Variables de Entorno

Variable	Ejemplo	Descripción
PORT	3000	Puerto en que escucha el servidor Express
DB_HOST	localhost	Host del servidor MySQL
DB_PORT	3306	Puerto del servidor MySQL
DB_USER	root	Usuario de la base de datos
DB_PASSWORD	***	Contraseña de la base de datos
DB_NAME	sistema_acuerdos	Nombre de la base de datos
JWT_SECRET	cadena aleatoria larga	Clave secreta para firmar tokens JWT
JWT_EXPIRES_IN	2h	Tiempo de expiración del token
UPLOADS_PATH	./uploads/actas	Ruta donde se almacenan los PDFs de actas

ADVERTENCIA: El archivo `.env` nunca debe incluirse en repositorios de control de versiones. Verificar que `.env` esté listado en el archivo `.gitignore` del proyecto.

10. Glosario

Término	Definición
API REST	Interfaz de programación de aplicaciones que sigue el estilo arquitectónico REST para comunicación HTTP.
JWT	JSON Web Token. Estándar para la transmisión segura de información como objeto JSON firmado digitalmente.
Soft Delete	Eliminación lógica. El registro se marca como eliminado pero permanece en la base de datos para trazabilidad.
Cron	Programador de tareas de Unix que permite ejecutar comandos en intervalos de tiempo definidos.
Pool de Conexiones	Conjunto de conexiones de base de datos reutilizables para mejorar el rendimiento bajo concurrencia.
Hash bcrypt	Algoritmo de hash adaptativo utilizado para almacenar contraseñas de forma segura.
SPA	Single Page Application. Aplicación web que carga una sola página HTML y actualiza el contenido dinámicamente.
Trigger	Procedimiento almacenado en MySQL que se ejecuta automáticamente ante eventos INSERT, UPDATE o DELETE.
Semáforo	Indicador visual de colores (ROJO/AMARILLO/VERDE) que refleja el estado de cumplimiento de un acuerdo.

Apéndice — Requisitos del Sistema

ID	Tipo	Descripción
RF01	Funcional	El sistema debe autenticar usuarios mediante correo y contraseña con generación de JWT.
RF02	Funcional	El sistema debe registrar y gestionar actas municipales con PDF adjunto.
RF03	Funcional	El sistema debe registrar acuerdos asociados a actas con plazo y estado.
RF04	Funcional	El sistema debe mostrar un semáforo visual por acuerdo (Pendiente/Cumplido/Vencido).
RF05	Funcional	El sistema debe mantener un historial de auditoría por acuerdo y global.
RF06	Funcional	El sistema debe gestionar usuarios con roles Administrador y Usuario.
RF07	Funcional	El sistema debe permitir adjuntar oficios PDF a los acuerdos.
RF08	Funcional	El sistema debe actualizar automáticamente acuerdos vencidos diariamente.
RNF01	No Funcional	Tiempo de respuesta menor a 3 segundos para consultas normales.
RNF02	No Funcional	Soporte mínimo de 10 usuarios concurrentes.
RNF03	No Funcional	Autenticación JWT con expiración configurable.
RNF04	No Funcional	Solo se permiten archivos en formato PDF, máximo 20 MB.
RNF05	No Funcional	Contraseñas almacenadas con hash bcrypt de 12 rondas.
RNF06	No Funcional	Disponibilidad del sistema en horario laboral municipal.